Date ___/___/_____

(*) JOINS:- A Join in SQL is used to combine data from two or more tables based on a related column between them.

(*) Syntax: SELECT column_names
FROM table1
JOIN table2
ON table1.common_column =
table2.common_column;

SELECT → Specifies the columns you want to display.
FROM → Specifies the first table.
JOIN → Specifies the 2nd table.
ON → Specifies the matching condition.

(*) Example - employees table :-

emp_id	emp_name	dept_id
1	Rahul	10
2	Priya	20
3	Divya	30
4	Kiran	40

departments table :-

dept_id	department
10	IT
20	HR
30	Finance
50	Marketing

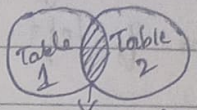
Here, dept_id is the common column in both tables. Using a JOIN, we can combine employee names with their department names.

```
SELECT employees.emp_name, departments.department
FROM employees
INNER JOIN departments
ON employees.dept_id = departments.dept_id;
```

← (If you don't want to use alias names for any type of join means)

(*) Types of SQL JOINS :-

1. INNER JOIN => Returns only the rows that have matching values in both tables.



Example:

(*) ~~Syntax~~: SELECT e.emp_name, d.department
FROM employees e
INNER JOIN departments d (or) JOIN departments d
ON e.dept_id = d.dept_id;

o/p :-

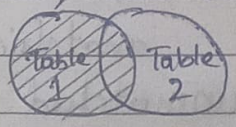
emp_name	department
Rahul	IT
Priya	HR
Divya	Finance

'e' & 'd' are table aliases (short names)

> Kiran is not displayed bcz dept_id = 40 doesn't exist in the dept table. ~~No~~

> Marketing is not displayed bcz no employee has dept_id = 50.

2. LEFT JOIN => Also called the Left Outer Join. Returns all rows from the left table & matching rows from the right table. If there is no match, the right table's columns contain NULL.
(Can use all comparitive operators)



(*) Example: SELECT e.emp_name, d.department
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id;

o/p -

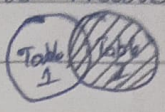
emp_name	department
Rahul	IT
Priya	HR
Divya	Finance
Kiran	NULL

(All employees are displayed, even Kiran who has no matching department)

(If you want All col^(n) to display in output) →
```
SELECT e.*, d.*
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.dept_id;
```

Saathi

3. RIGHT JOIN ⇒ Also called Right Outer Join. Returns all rows from the right table & matching rows from the left table. If there is no match, the left table's columns contain NULL.



(*) Example -
```
SELECT e.emp_name, d.department
FROM employees e
RIGHT JOIN departments d
ON e.dept_id = d.dept_id;
```

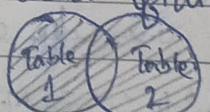
~~o/p~~ - (Returns all records from the right table i.e, departments & matching records from the left table i.e, employees)

o/p -

emp_name	department
Rahul	IT
Priya	HR
Divya	Finance
NULL	Marketing

Marketing appears even though no employee belongs to dept 50.

4. Full Outer Join ⇒ Returns all rows from both tables, matching them where possible and filling unmatched columns with NULL.



(*) Example :
```
SELECT e.emp_name, d.department
FROM employees e
LEFT JOIN departments d
ON e.dept_id = d.dept_id;
UNION
SELECT e.emp_name, d.department
FROM employees e
```

(Or if you want to display all columns then write e.*, d.*)

Saathi

```
RIGHT JOIN departments d
ON e.dept_id = d.dept_id;
```

5. CROSS JOIN ⇒ Combines every row from the first table with every row from the second table. Returns every possible combination of rows from both tables.

table 1	table 2
id 1	id 2
1	A
2	B
3	C

Output →

CROSS JOIN	
id1	id2
1	A
1	B
1	C
2	A
2	B
2	C
3	A
3	B
3	C

(2) Example:

```
SELECT e.emp_name, d.department
FROM employees e
CROSS JOIN departments d;
```

> cross join doesn't require an 'ON' condition bcz it displays all valid & invalid data.

Output :-

emp_name	department
Rahul	IT
Rahul	HR
Rahul	Finance
Rahul	Marketing
Priya	IT
Priya	HR
Priya	Finance
Priya	Marketing
Divya	IT
Divya	HR
Divya	Finance
Divya	Marketing
Kiran	IT
Kiran	HR
Kiran	Finance
Kiran	Marketing

Date __/__/____

Saathi

6. Self Join ⇒ Joins a table with itself to compare or connect rows with same table.

for example, suppose the employees table contains :-

emp_id	emp_name	manager_id
1	Rahul	NULL
2	Priya	1
3	Divya	1
4	Kiran	2

It is useful when comparing rows in the same table, such as employees & their managers.

(*) Example - (1) SELECT e.emp_name AS Employee,
~~m. manag~~ m.emp_name AS Manager
FROM employees e
JOIN employees m
ON e.manager_id = m.emp_id;

o/p -

Employee	Manager
Priya	Rahul
Divya	Rahul
Kiran	Priya

Here, Rahul is not displaye bcz he has no matching manager record.

(2) SELECT e.emp_name AS Employee,
m.emp_name AS Manager
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.emp_id;

o/p -

Employee	Manager
Rahul	NULL
Priya	Rahul
Divya	Rahul
Kiran	Priya

Here, Rahul has no manager, but he is still displayed because we used LEFT JOIN.

Page No.

(*) What is the difference?

JOIN (INNER JOIN)	LEFT JOIN
> Returns only matching rows.	> Returns all rows from the left side, including unmatched rows.
> Rahul is excluded.	> Rahul is included with NULL as his manager.

(*) Remember ⇒ "Self Join" describes joining a table with itself. It doesn't require a specific JOIN type.
You can use JOIN, LEFT JOIN, or even RIGHT JOIN depending on the condition.
So, if you want only employees who have a matching manager, use JOIN.
If you want all employees, including those without a manager, use LEFT JOIN.

(*) Example for Right Join in Self Join -

(3.)

emp_id	emp_name	manager_id
1	Rahul	NULL
2	Priya	1
3	Divya	1
4	Kiran	2

```
SELECT e.emp_name AS Employee
       m.emp_name AS Manager
RIGHT FROM employees e
RIGHT JOIN employees m
ON e.manager_id = m.emp_id;
```

RIGHT JOIN - preserves every row from the right side table (m), even when there is no matching employee on the left side (e).

Saathi

Date ___/___/_____

O/p –

Employee	Manager
Priya	Rahul
Divya	Rahul
Kiran	Priya
NULL	Divya
NULL	Kiran

> When is "Right Join" Useful in a "Self Join"?
You can use it when you want to display all employees in the manager role, including those who do not manage anyone.
In a "Self Join", both sides refer to the same physical table, but their aliases (e and m) let you treat them as 2 separate roles.

(*) Differences:-

Term	Meaning
(1) JOIN	Usually means INNER JOIN.
(2) INNER JOIN	Returns matching rows.
(3) SELF JOIN	Joins a table with itself.
(4) SELF JOIN using 'JOIN'	Joins a table with itself & returns matching rows.
(5) SELF JOIN using 'LEFT JOIN'	Joins a table with itself & preserves all rows from the left side.
(6) SELF JOIN using 'RIGHT JOIN'	Joins a table with itself & preserves all rows from the right side.

Page No.